

CSE 144 Final Project Report

Transfer Learning Challenge

Francisco Martinez (CruzID: 2171233)

Spring 2026

1 Introduction

In this project, the goal is to build an image classifier for a dataset of 100 classes where each class contains around 10 training images (total of around 1,000 images). Due to this limited dataset, building an image classifier from scratch would not be practical. Training an image classifier like the simple CNN used in earlier assignments will lead to underfitting as we have 100 classes or lead to overfitting due to limited data size.

This problem can be solved by transfer learning. By using a backbone network which has already been trained on ImageNet, it will have already learned general features like edges, textures, and object parts from over a million images. Now by fine-tuning it on our limited dataset, we can use those learned features but still adapt our network to the specific problem we are dealing with.

We have fine-tuned an EfficientNet-B3 network which was pre-trained on ImageNet, used dropout regularization for the classification head, learning rate decay, label smoothing, and data augmentation. The results were a 68.98% validation accuracy on a test split of the dataset and 0.763 on the Kaggle leaderboard which exceeds our benchmark of 60%.

2 Dataset

There are 100 classes taken from various sources. Every class has 10 training images, meaning there are 1,000 training images total and 1,000 images in the test set (unlabeled). The training set has an 80/20 split into training and validation sets (800 vs 200 images), respectively. This partitioning is performed randomly using the same randomization sequence generated with seed=42.

In terms of the filesystem structure, train/ has 100 directories named "0" through "99." They contain approximately 10 images each and represent 100 different classes. test/ has 1,000 images named from 0.jpg to 999.jpg. Since torchvision.datasets.ImageFolder enumerates directories lexicographically as string labels ("0", "1", "10", "100", ...), the internal representation does not match real class labels. To fix that issue, we create an explicit mapping from ImageFolder's class_to_idx to integer labels corresponding to those folder names for every prediction. Otherwise, the ordering would have been incorrect, and the performance scores would be random.

Image sizes vary; hence, all images are resized to 300x300px and normalized based on ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). During training, we perform the following data augmentations: random crop of size 300px (scale: 0.75-1.0), random flips, random rotations up to 15 degrees, and color jitter (brightness/contrast/saturation 0.2). During validation and test time, the only transforms are resizing and normalization.

3 Implementation

3.1 Model

EfficientNet-B3 trained on ImageNet (torchvision EfficientNet_B3_Weights.DEFAULT) serves as our backbone. One of the advantages of using EfficientNet-B3 is its good trade-off between model accuracy and computational cost: it gives enough accuracy to do transfer learning and have enough parameters (~12 million) not to get overfitted on the training set, which consists of only 1,000 images compared to other models.

The original head is substituted by a custom one as follows:

Dropout(0.4) → Linear(1536, 512) → SiLU → Dropout(0.3) → Linear(512, 100)

Here the two dropout layers form the key protection from getting overfitted on the dataset. The entire neural network (the backbone plus the head) is fine-tuned end-to-end without freezing any layers but using different learning rates instead. The backbone gets a learning rate ten times smaller than that of the randomly initialized head.

3.2 Training

- Loss: cross-entropy with label smoothing 0.1
- Optimizer: AdamW, weight decay 1e-4; backbone LR 1e-4, classifier head LR 1e-3
- Scheduler: cosine annealing over 30 epochs ($T_{\text{max}} = 30$, $\eta_{\text{min}} = 1\text{e-}6$)
- Batch size: 32; epochs: 30; input resolution: 300×300
- Model selection: checkpoint with the best validation accuracy is saved and used for inference
- Hardware/software: Google Colab, NVIDIA Tesla T4 GPU, PyTorch with torchvision

4 Experiments

Baseline. Baseline architecture was the same EfficientNet-B0, but now with the dropout classifier head. This network achieved a score of 0.627 on Kaggle, slightly higher than the 60% baseline, thus establishing that the data handling process and all other aspects of the pipeline are sound.

Hyperparameter tuning. All hyperparameters were tuned using the holdout validation set, while the test set remained untouched until the end, serving solely for Kaggle submissions. Variation proceeded systematically, one parameter at a time: backbone capacity (B0 vs. B3), regularizer (linear head vs. dropout head), and augmentation technique (normal augmentations vs. mixup).

Ablation results. The biggest gain was obtained by going from B0 to B3, corroborating the hypothesis that better pretrained representations yield the most significant gains when the amount of data is limited. Adding the dropout head to B3 resulted in yet another boost, whereas using the B3 architecture without dropout together with mixup barely exceeded 60%. Thus, we

can state that when there are only 10 images per class, mixing labels using mixup is too aggressive. Too aggressive calls for overfit.

5 Results

Final results for the model variants we trained:

Model	Validation Accuracy (%)	Kaggle Public Score
EfficientNet-B0 + dropout head	~60	0.627
EfficientNet-B3 (plain head)	~65	—
EfficientNet-B3 + mixup	~60	—
EfficientNet-B3 + dropout head (final)	68.98	0.763

Final EfficientNet-B3 + dropout model scored 68.98% validation accuracy, 0.763 on the public Kaggle leaderboard (around 10% of test set). Gap between validation and public is expected, since validation split consists of only ~2 images per class (total 200).

6 Discussion

Approaches that worked well. Combination of moderate-sized pretrained backbone (B3), strong dropout regularization in the head, discriminative learning rates, label smoothing, and cosine learning-rate decay. Each part helps with the main challenge in this task: overfitting the relatively small 1,000 image dataset.

Approaches that didn't help. Mixup actually hurt performance instead of improving it; interpolating between classes with only 10 examples each eliminates most of the label signal. Plain non-dropout B3 head overfit; training loss was very far from validation loss.

Improvements. Current model was trained on 800 images only (remaining 200 used for validation), but training on entire dataset with selected hyperparameters should yield better performance. Practical improvements include using larger backbones (such as ViT-B/16 with SWAG weights, or EfficientNetV2-S), averaging predictions over horizontally flipped images (test-time augmentation), and ensemble averaging multiple backbones' output logits.

7 Reproducibility

- Random seed 42 is initialized for Python's random library, NumPy, and PyTorch (both CPU & CUDA). Split into train/validation sets is done via a deterministic generator.
- Used environment: Google Colab notebook with NVIDIA Tesla T4 GPU, PyTorch and torchvision in their default Colab version. There may be some variance in results due to cuDNN non-deterministic mode.
- Training procedure. Load Google Colab GPU runtime, load competition data as .zip file to /content/ folder, and execute all notebook cells sequentially (takes about 30-40 min).

- Creating submission.csv without training. Download weights best_effb3_dropout.pth from provided Google Drive link in README.md, upload it to /content/ folder, execute cells from "model.load_state_dict" onward. Saves submission_effb3_dropout.csv file.

8 Team Contributions

All of the code, experiments, writing, and the presentation were done by me.

9 References

- TorchVision pretrained models documentation: <https://pytorch.org/vision/stable/models.html>
- M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML 2019.